

*{Módulo que implementa el TAD Diccionario con representación interna basada en árbol ABB en memoria dinámica.*

*Versión: 1.0*

*Fecha: 2019-11-14 (en 2020-11 añadida operación Iguales, última actualización 2024-10-28)*

*Autor: Yolanda Villate}*

módulo genérico diccionariosABB

usa pilasGenéricas *{implementación dinámica, de la lección 9}*

parámetros

tipos clave,valor

con

función "<" (c1,c2:clave) devuelve booleano*{devuelve verdad sii c1 es menor que c2}*

función "=" (c1,c2:clave) devuelve booleano*{devuelve verdad sii c1 es igual que c2}*

exporta

tipo diccionario *{Documentación a completar...}*

procedimiento crear(sal d:diccionario);

*{Documentación a completar...}*

función cardinal(d:diccionario) devuelve entero;

*{Documentación a completar...}*

función esVacio?(d:diccionario) devuelve booleano;

*{Documentación a completar...}*

procedimiento añadir(e/s d:diccionario; ent c:clave; ent v:valor);

*{Documentación a completar...}*

procedimiento buscar(ent d:diccionario; ent c:clave; sal éxito:booleano; sal v:valor);

*{Documentación a completar...}*

*{pertenece? y obtenerValor se implementan en una única operación}*

procedimiento quitar(ent c:clave; e/s d:diccionario);

*{Documentación a completar...}*

*{Operaciones de Iterador interno, para recorrer todos los datos del diccionario en orden por clave de menor a mayor:}*

procedimiento iniciarIterador(e/s d:diccionario);

*{Documentación a completar...}*

función existeSiguiente?(d:diccionario) devuelve booleano;

*{Documentación a completar...}*

procedimiento siguienteYavanza(e/s d:diccionario; sal c:clave; sal v:valor;  
sal error:booleano);

*{Documentación a completar...}*

*{Operaciones para duplicar y liberar:}*

procedimiento duplicar(sal dSal:diccionario; ent dEnt:diccionario);

*{Crea una copia (profunda) de la colección de pares (clave,valor) del diccionario dEnt en el diccionario dSal.}*

procedimiento liberar(e/s d:diccionario);

*{Libera toda la memoria utilizada por el diccionario, y deja el diccionario vacío}*

función iguales?(d1,d2:diccionario) devuelve booleano

*{Devuelve verdad si y solo si el diccionario d1 y el diccionario d2 tienen exactamente la misma colección de pares (clave,valor).}*

implementación

```

tipos      {Documentación a completar...}
arbin = ↑nodo;
elemento = registro
            laClave:clave;
            elValor: valor

freg
nodo = registro
        dato:elemento;
        izq,der:arbin
freg

módulo pila_de_arbin concreta pilasGenéricas(arbin);

diccionario=registro
    cardinal: entero;
    raíz: arbin; {puntero a la raíz de un árbol binario}
    iter: pila_de_arbin.pila
        {Para el iterador: pila de punteros a nodos del árbol binario}
freg

{Documentación a completar...}
procedimiento crear(sal d:diccionario)
principio
    crear(d.iter); {inicializar a vacía la pila del iterador de diccionario d}
    d.cardinal:= 0;
    d.raíz=nil
fin

{Documentación a completar...}
función cardinal(d:diccionario) devuelve entero
principio
    devuelve d.cardinal
fin

{Documentación a completar...}
función esVacio?(d:diccionario) devuelve booleano
principio
    devuelve (d.raíz=nil)
fin

{Documentación a completar...}
procedimiento añadir(e/s d:diccionario; ent c:clave; ent v:valor)
    variable añadido:booleano
principio
    insertar(d.raíz,c,v,añadido);
    si añadido entonces
        d.cardinal:=d.cardinal+1
    fsi
fin

{OPERACIÓN PRIVADA: } {Documentación a completar...}
procedimiento insertar(e/s a: arbin; ent c:clave; ent v:valor; sal añadido: booleano)
principio
    si a=nil entonces
        nuevoDato(a);
        a↑.dato.laClave:=c;
        a↑.dato.elValor:=v;
        a↑.izq:=nil;
        a↑.der:=nil;
        añadido:=verdad

```

```

sino
    si c<a↑.dato.laClave entonces
        insertar(a↑.izq,c,v,añadido)
    sino_si a↑.dato.laClave<c entonces
        insertar(a↑.der,c,v,añadido)
    sino
        a↑.dato.elValor:=v;
        añadido:=falso
    fsi
fsi
fin

{Documentación a completar...}
procedimiento buscar(ent d:diccionario; ent c:clave; sal éxito:booleano; sal v:valor)
    {pertenece? y obtenerValor se implementan en una única operación}
principio
    está?(d.raíz,c,éxito,v);
fin

{OPERACIÓN PRIVADA: } {Documentación a completar...}
procedimiento está?(ent a: arbin; ent c:clave; sal éxito:booleano; sal v:valor)
principio
    si a=nil entonces
        éxito:=falso
    sino
        selección
            c<a↑.dato.laClave: está?(a↑.izq,c,éxito,v);
            c=a↑.dato.laClave: éxito:=verdad;
                                v:= a↑.dato.elValor;
            a↑.dato.laClave<c: está?(a↑.der,c,éxito,v)
        fselección
    fsi
fin

{Documentación a completar...}
procedimiento quitar(ent c:clave; e/s d:diccionario)
    variable quitado:booleano
principio
    borrar(d.raíz,c,quitado);
    si quitado entonces
        d.cardinal:=d.cardinal-1
    fsi
fin

{OPERACIÓN PRIVADA: } {Documentación a completar...}
procedimiento borrarMaxClave(e/s a:arbin; sal e:elemento)
    {Dado un árbol no vacío a (a≠nil), devuelve en e el máximo elemento del árbol a, y además
    elimina ese elemento del árbol a}
    variable aux:arbin;
principio
    si a↑.der=nil entonces
        e:=a↑.dato;
        aux:=a;
        a:=a↑.izq;
        disponer(aux)
    sino
        borrarMaxClave(a↑.der,e)
    fsi
fin

```

```

{OPERACIÓN PRIVADA: } {Documentación a completar...}
procedimiento borrar(e/s a:arbin; ent c: clave; sal eliminado: booleano)
  variable aux: arbin;
principio
  si a≠nil entonces
    selección
      c<a↑.dato.laClave: borrar(a↑.izq,c);
      a↑.dato.laClave<c: borrar(a↑.der,c);
      c=a↑.dato.laClave: eliminado:=verdad;
      si a↑.izq=nil entonces
        aux:=a; a:=a↑.der; disponer(aux)
      sino_si a↑.der=nil entonces
        aux:=a; a:=a↑.izq; disponer(aux)
      sino {ambos hijos son no vacíos}
        borrarMaxClave(a↑.izq,a↑.dato)
      fsi
    fselección
  sino
    eliminado:=falso {el dato a borrar no se ha encontrado}
  fsi
fin

{Documentación a completar...}
procedimiento iniciarIterador(e/s d: diccionario)
  variable aux:arbin
principio
  {eliminar restos de recorridos anteriores (no exhaustivos) y empezar con pila vacía}
  liberar(d.iter);
  aux:=d.raíz; {raíz del árbol}
  mq aux≠nil hacer
    apilar(d.iter,aux); {apila puntero a nodo del árbol, en la pila del iterador}
    aux:=aux↑.izq
  fmq
fin

{Documentación a completar...}
función haySiguiente(d:diccionario) devuelve booleano
principio
  devuelve not esVacía(d.iter) {hay siguiente si la pila del iterador es no vacía}
fin

{Documentación a completar...}
procedimiento siguienteYavanza(e/s d:diccionario; sal c:clave; sal v:valor;
                              sal error:booleano)
  variable aux:arbin
principio
  si haySiguiente(d) entonces
    error:=falso;
    aux:=cima(d.iter);
    c:=aux↑.dato.laClave; {esta es la clave del siguiente elemento visitado}
    v:=aux↑.dato.elValor; {este es el valor del siguiente elemento visitado}
    {avanzar: }
    desapilar(d.iter);
    aux:=aux↑.der;
    mq aux≠nil hacer
      apilar(d.iter,aux); aux:=aux↑.izq
    fmq
  sino
    error:=verdad
  fsi
fin

```

*{Se crea una copia profunda del diccionario dEnt en el diccionario dSal, es decir se copia la información de todos sus pares (clave,valor). Además se copia la forma del árbol que contiene en sus nodos los pares (clave, valor), pero no se copia el estado del iterador para recorrer el árbol del diccionario dEnt (es decir, la pila usada por las operaciones del iterador), ya que eso no es información de la colección de pares (clave,valor) del diccionario (y además, si copiásemos directamente la pila de su iterador, estaríamos copiando punteros a nodos del árbol del diccionario dEnt).}*

**procedimiento** duplicar(**sal** dSal:diccionario; **ent** dEnt:diccionario)

**principio**

    crear(dSal.iter); *{inicializar a vacía la pila del iterador de diccionario dSal}*

    dSal.cardinal:= dEnt.cardinal;

    duplicarArbol(dSal.raíz, dEnt.raíz);

**fin**

*{OPERACIÓN PRIVADA: } {Documentación a completar...}*

**procedimiento** duplicarArbol(**sal** nuevo:arbin; **ent** viejo:arbin)

**variables** ai,ad:arbin

**principio**

**si** viejo=nil **entonces**

        nuevo:=nil

**sino**

        duplicarArbol (ai,viejo↑.izq);

        duplicarArbol (ad,viejo↑.der);

        nuevoDato(nuevo);

        nuevo↑.dato:=viejo↑.dato;

        nuevo↑.izq:=ai;

        nuevo↑.der:=ad

**fsi**

**fin**

*{Libera toda la memoria utilizada por el diccionario, y deja en d diccionario vacío}*

**procedimiento** liberar(**e/s** d:diccionario)

**principio**

    liberar(d.iter); *{o bien: pila\_de\_arbin.liberar(d.iter)}*

    liberarRec(d.raíz);

    d.cardinal:=0;

**fin**

*{OPERACIÓN PRIVADA:*

*Libera toda la memoria utilizada por el árbol a, en el que se almacenan los datos del diccionario, y deja en a el árbol vacío}*

**procedimiento** liberarRec(**e/s** a:arbin)

**principio**

**si** a ≠nil **entonces**

        liberarRec(a↑.izq);

        liberarRec(a↑.der);

        disponer(a);

        a:=nil

**fsi**

**fin**

```

{Si se añadiese la operación de Iguales:}
función iguales?(d1,d2:diccionario) devuelve booleano
variables igual:booleano;
    {y para recorrerlos simultáneamente en inorden:}
    pAux1,pAux2:arbin; pilaD1,pilaD2:pila_de_arbin.pila;
principio
si esVacio?(d1) and esVacio?(d2) entonces devuelve verdad
sino_si cardinal(d1)≠cardinal(d2) entonces devuelve falso
sino {ambos tienen el mismo número (no nulo) de claves: recorrerlos simultáneamente en
orden por clave (inorden)}
    igual:= verdad;
    {inicializar recorrido en diccionario d1:}
    {Llegar hasta el primero y prepararlo para que sea el siguiente a tratar:}
    crear(pilaD1);
    pAux1:=d1.raíz;
    mq pAux1≠nil hacer
        apilar(pilaD1,pAux1);    pAux1:=pAux1↑.izq
    fmq;
    {Ídem en diccionario d2:}
    crear(pilaD2);
    pAux2:=d2.raíz;
    mq pAux2≠nil hacer
        apilar(pilaD2,pAux2);    pAux2:=pAux2↑.izq
    fmq;
    {Mientras sean iguales y no se hayan tratado todos:}
    mientrasQue igual and not esVacía(pilaD1) and not esVacía(pilaD2) hacer
        {Obtener el siguiente que toca tratar de cada diccionario: }
        pAux1:=cima(pilaD1);    pAux2:=cima(pilaD2);
        {Compararlos: }
        igual:=(pAux1↑.dato.laClave=pAux2↑.dato.laClave) and
            (pAux1↑.dato.elValor=pAux2↑.dato.elValor);

        {Avanzar, para preparar el siguiente dato a tratar en diccionario d1:}
        desapilar(pilaD1);
        pAux1:=pAux1↑.der;
        mq pAux1≠nil hacer
            apilar(pilaD1,pAux1);    pAux1:=pAux1↑.izq
        fmq

        {Ídem en diccionario d2:}
        desapilar(pilaD2);
        pAux2:=pAux2↑.der;
        mq pAux2≠nil hacer
            apilar(pilaD2,pAux2);    pAux2:=pAux2↑.izq
        fmq

    fmq;
    si not igual entonces
        liberar(pilaD1);
        liberar(pilaD2)
    fsi;
    devuelve igual
fsi
fin

fin //del módulo

```